so I have a problem where i am rate limited by a website when i make more than 20 rqurests per minute. should i create a function called by my request function to check that i am not breaking this, and has a cool down feature if so

Yes, implementing a function to monitor and enforce the rate limit is a great approach. Here's a strategy:

1. **Track Requests**: Maintain a timestamp log of requests.
2. **Check Rate Limit**: Before making a request, check if you have already sent 20 requests in the last minute.
3. **Cooldown Handling**: If the limit is exceeded, wait until you can send the next request.
4. **Thread Safety** (Optional): If using multithreading or async requests, use a thread-safe mechanism like `threading.Lock` or `asyncio`.

Here's an example implementation using Python with `time`:

Here's an example implementation using Python with `time`:

python Copy Edit

```python
import time
from collections import deque

class RateLimiter:
    def __init__(self, max_requests=20, time_window=60):
        self.max_requests = max_requests
        self.time_window = time_window  # In seconds
        self.request_timestamps = deque()  # Stores timestamps of requests

    def allow_request(self):
        """Checks if a request is allowed, and enforces cooldown if needed."""
        current_time = time.time()

        # Remove old timestamps outside the time window
        while self.request_timestamps and self.request_timestamps[
```

no need to create a new class, it'll just be a function within my class already

Got it! You can integrate a function within your existing class to handle rate limiting. Here's a simple function that ensures you don't exceed 20 requests per minute and enforces a cooldown if needed:

Implementation:

python Copy Edit

```python
import time
from collections import deque

class YourClass:
    def __init__(self):
        self.request_timestamps = deque()  # Track request timestamps
        self.max_requests = 20
        self.time_window = 60  # 60 seconds

    def _check_rate_limit(self):
        """Ensures that requests don't exceed the limit. If exceeded, it waits."""
        current_time = time.time()

        # Remove timestamps older than the time window
        while self.request_timestamps and self.request_timestamps[0] < current_time
            self.request_timestamps.popleft()
```

ChatGPT can make mistakes. Check important info.

Implementation:

python Copy Edit

```python
import time
from collections import deque

class YourClass:
    def __init__(self):
        self.request_timestamps = deque()  # Track request timestamps
        self.max_requests = 20
        self.time_window = 60  # 60 seconds

    def _check_rate_limit(self):
        """Ensures that requests don't exceed the limit. If exceeded, it waits."""
        current_time = time.time()

        # Remove timestamps older than the time window
        while self.request_timestamps and self.request_timestamps[0] < current_tim
            self.request_timestamps.popleft()

        # If the limit is reached, sleep until we can send the next request
        if len(self.request_timestamps) >= self.max_requests:
            wait_time = self.request_timestamps[0] + self.time_window - current_ti
            print(f"Rate limit exceeded. Sleeping for {wait_time:.2f} seconds...")
            time.sleep(wait_time)

        # Log the new request timestamp
        self.request_timestamps.append(time.time())

    def make_request(self):
        """Example function that calls _check_rate_limit before sending a request."
        self._check_rate_limit()
        # Your actual request logic goes here
        print("Request made")

# Example Usage
obj = YourClass()
for _ in range(25):  # Simulating 25 requests
    obj.make_request()
```

How It Works:

1. **Tracks timestamps** of requests in a `deque` (efficient queue).
2. **Removes old timestamps** that fall outside the 60-second window.
3. **Checks the request count** in the window:
   - If within limits → Proceeds immediately.
   - If exceeded → Sleeps until the oldest request expires.
4. **Logs the new request timestamp** after making a request.

This ensures you never exceed 20 requests per minute while keeping the code simple. 🚀

how do I have a package that I can use internally?

If you want to create a package that you can use internally within your organization or project (without publishing it to public repositories like PyPI), here's how you can do it:

---

1. Create the Package Structure

Organize your package with a proper directory structure:

arduino Copy Edit

```arduino
my_package/
|— my_package/
|   |— __init__.py
|   |— module1.py
|   |— module2.py
|— setup.py
|— pyproject.toml  # (Optional, for modern builds)
|— README.md
|— requirements.txt
|— setup.cfg  # (Optional, for easier setup)
|— tests/
|   |— test_module1.py
```

- `__init__.py`: Makes it a package.
- `module1.py`, `module2.py`: Your actual code files.
- `setup.py`: Defines installation settings.
- `requirements.txt`: Dependencies.
- `tests/`: Unit tests.

---

2. Write `setup.py` for Installation

Create a `setup.py` file for installing the package:

python Copy Edit

```python
from setuptools import setup, find_packages

setup(
    name="my_package",
    version="0.1.0",
    packages=find_packages(),
    install_requires=[
        # List dependencies here
    ],
    author="Your Name",
    description="An internal packag or our organization",
)
```

TL;DR

✅ A directory with an `__init__.py` file = a package
✅ You can import it directly if it's in your project
✅ If outside, add it to `sys.path` or install it locally

Would you like help structuring your package for long-term use? 🚀

2/2

```
UNWIND $matches AS match_data
CREATE (match:Match {id: match_data.Match_ID})
SET match.date = date(match_data.Date),
   match.time = time(match_data.Time),
   match.home_score = CASE WHEN match_data.Match_Report
THEN toInteger(match_data.Home_Score) ELSE NULL END,
   match.away_score = CASE WHEN match_data.Match_Report
THEN toInteger(match_data.Away_Score) ELSE NULL END,
   match.home_expected = CASE WHEN
match_data.home_expected IS NOT NULL THEN
toFloat(match_data.Home_Expected) ELSE NULL END,
   match.away_expected = CASE WHEN
match_data.home_expected IS NOT NULL THEN
toFloat(match_data.Away_Expected) ELSE NULL END,
   match.game_week = match_data.GW,
   match.attendance = match_data.Attendance

MERGE (venue:Venue {name: match_data.Venue})
MERGE (match)-[:PLAYED_AT]->(venue)

WITH match, match_data
MATCH (home_team:Squad {team: match_data.Home,
season:$season_name})
MATCH (away_team:Squad {team: match_data.Away, season:
$season_name})
MERGE (season:Season {season:$season_name})
MERGE (match)-[:HOME_TEAM]->(home_team)
MERGE (match)-[:AWAY_TEAM]->(away_team)
MERGE (match)-[:IS_PART_OF]->(season)

// Conditionally create the referee node only if the referee data exists
WITH match, match_data
WHERE match_data.Referee IS NOT NULL
MERGE (referee:Referee {name: match_data.Referee})
MERGE (match)-[:HAS_REFEREE]->(referee)

// Ensure home and away scores and expectations are set correctly
SET match.home_score = CASE WHEN match_data.Home_Score IS
NOT NULL THEN toInteger(match_data.Home_Score) ELSE
match.home_score END,
   match.away_score = CASE WHEN match_data.Away_Score IS
NOT NULL THEN toInteger(match_data.Away_Score) ELSE
match.away_score END,
   match.home_expected = CASE WHEN
match_data.Home_Expected IS NOT NULL THEN
toFloat(match_data.Home_Expected) ELSE match.home_expected
END,
   match.away_expected = CASE WHEN match_data.Away_Expected
IS NOT NULL THEN toFloat(match_data.Away_Expected) ELSE
match.away_expected END i currently have this query where the
logic is slightly weak, can we do it more dynamically so that in the
case when the data has an attribute Match_Report == True set the
home and away score, same for referee, the referee relation: has
referee etc. this depends on each data entry so its on a case by case
basis
```

Here's a more dynamic version of your Cypher query that ensures attributes and relationships are only created when necessary, making it more robust:

cypher

Copy Edit

```cypher
UNWIND $matches AS match_data

// Create match node with mandatory attributes
CREATE (match:Match {id: match_data.Match_ID})
SET match.date = date(match_data.Date),
    match.time = time(match_data.Time),
    match.game_week = match_data.GW,
    match.attendance = match_data.Attendance

// Conditionally set match scores only if Match_Report is true
FOREACH (_ IN CASE WHEN match_data.Match_Report THEN [1] ELSE [] END |
    SET match.home_score = toInteger(match_data.Home_Score),
        match.away_score = toInteger(match_data.Away_Score)
)

// Conditionally set expected scores if values are present
FOREACH (_ IN CASE WHEN match_data.Home_Expected IS NOT NULL THEN [1] ELSE [] END
    SET match.home_expected = toFloat(match_data.Home_Expected)
)
FOREACH (_ IN CASE WHEN match_data.Away_Expected IS NOT NULL THEN [1] ELSE [] END
    SET match.away_expected = toFloat(match_data.Away_Expected)
)

// Create venue and relationship
MERGE (venue:Venue {name: match_data.Venue})
MERGE (match)-[:PLAYED_AT]->(venue)

// Match teams
WITH match, match_data
MATCH (home_team:Squad {team: match_data.Home, season: $season_name})
MATCH (away_team:Squad {team: match_data.Away, season: $season_name})

// Create season relationship
MERGE (season:Season {season: $season_name})
MERGE (match)-[:HOME_TEAM]->(home_team)
MERGE (match)-[:AWAY_TEAM]->(away_team)
MERGE (match)-[:IS_PART_OF]->(season)

// Conditionally create referee relationship
FOREACH (_ IN CASE WHEN match_data.Referee IS NOT NULL THEN [1] ELSE [] END |
    MERGE (referee:Referee {name: match_data.Referee})
    MERGE (match)-[:HAS_REFEREE]->(referee)
)
```

Key Improvements:

1. **More Dynamic Attribute Assignment**:
   - Uses `FOREACH` with `CASE WHEN` to conditionally set `home_score`, `away_score`, and expected values only when valid.
2. **Conditional Relationships with `FOREACH`**:
   - The `HAS_REFEREE` relationship is o. created if `match_data.Referee` is not `NULL`.

Your script creates an interactive UI in a Jupyter Notebook that connects to a Neo4j database, retrieves available seasons, and allows users to fetch team rankings based on match results. Here are a few key observations and possible improvements:

Strengths:

1. **Modular Functions**:
   - Well-structured functions for fetching unique seasons, teams, and match data.
   - Queries are optimized with pattern matching in Neo4j.
2. **Interactive UI**:
   - Uses `ipywidgets` to make selection and data retrieval interactive.
   - Displays results in a Pandas DataFrame for easy visualization.
3. **Clear Data Processing**:
   - Uses `UNWIND` to process both home and away team points efficiently.

Suggestions for Improvement:

1. **Fix `fetch_data` DataFrame Issue**:
   The `fetch_data` function creates a DataFrame directly from the result set, but `record for record in result` returns a list of `neo4j.Record` objects instead of dictionaries. Instead, you should explicitly extract values:

can you change it from a dropdown to a button please

Sure! Here's the modified code where the dropdown is replaced with a button. The user will now click a button to fetch the list of seasons instead of selecting from a dropdown.

Neo4j Ui Button

Now, instead of a dropdown, the UI has:

1. A **"Fetch Seasons"** button that retrieves available seasons.
2. It **automatically selects the latest season** and enables the "Fetch Data" button.
3. Clicking **"Fetch Data"** retrieves the match rankings for the selected season.

Let me know if you want any further adjustments! 🚀

can you remove the fetch button please

Neo4j Ui Button

```
import os
from neo4j import GraphDatabase
import pandas as pd
```

```
import os
from neo4j import GraphDatabase
import pandas as pd
from ipywidgets import VBox, Button, Dropdown, Output
from IPython.display import display
from dotenv import load_dotenv

env_path = os.path.join(os.getcwd(),'config/.env')
load_dotenv(dotenv_path=env_path)

NEO4J_URI = os.getenv("NEO4J_URI")
NEO4J_USERNAME = os.getenv("NEO4J_USERNAME")
NEO4J_PASSWORD = os.getenv("NEO4J_PASSWORD")

driver = GraphDatabase.driver(NEO4J_URI, auth=
(NEO4J_USERNAME, NEO4J_PASSWORD))

# Functions for fetching data
def get_unique_seasons():
    query = "MATCH (s:Season) RETURN s.name AS season_name
ORDER BY season_name"
    with driver.session(database="neo4j") as session:
        result = session.run(query)
        return [record['season_name'] for record in result]


def get_unique_teams(season_name):
    query = """
    MATCH (squad:Squad)
    WHERE squad.name CONTAINS $season
    RETURN squad.name AS squad_name
    """
    with driver.session(database="neo4j") as session:
        result = session.run(query, parameters={'season':
season_name})
        return [
            record['squad_name'].replace(season_name, "").strip()
            for record in result
        ]


def fetch_data(season_name):
    query = """
    MATCH (match:Match)-[:IS_OF_GW]->(gw:GameWeek {season:
$season_name})
    MATCH (match)-[:HOME_TEAM]->(home_team:Squad)
    MATCH (match)-[:AWAY_TEAM]->(away_team:Squad)
    WITH
        home_team.name AS team,
        CASE
            WHEN match.home_score > match.away_score THEN 3
            WHEN match.home_score = match.away_score THEN 1
            ELSE 0
        END AS home_points,
        away_team.name AS away_team,
        CASE
            WHEN match.away_score > match.home_score THEN 3
```

Neo4j Ui Button

```python
import os
from neo4j import GraphDatabase
import pandas as pd
from ipywidgets import VBox, Button, Output
from IPython.display import display
from dotenv import load_dotenv

# Load environment variables
env_path = os.path.join(os.getcwd(), 'config/.env')
load_dotenv(dotenv_path=env_path)

# Neo4j Connection Details
NEO4J_URI = os.getenv("NEO4J_URI")
NEO4J_USERNAME = os.getenv("NEO4J_USERNAME")
NEO4J_PASSWORD = os.getenv("NEO4J_PASSWORD")
```

Edited

I've removed the fetch button and modified the season fetch button so that it automatically retrieves data when a season is selected. Let me know if you need any more changes! 🚀

my json file has Nan in it but cant be written, it seems like json.dump doesn't support this, what do?

Use `simplejson`! It supports `NaN` handling with `ignore_nan=True`, which will remove or replace `NaN` values automatically:

python Copy Edit

```python
import simplejson as json
import numpy as np

data = {
    "value1": 123,
    "value2": np.nan  # This is a NaN
}

with open("output.json", "w") as f:
    json.dump(data, f, ignore_nan=True)  # Removes NaN values
```

This will ensure that `NaN` values are either omitted or converted to `null`, making your JSON valid. 🚀